



← All roadmaps

MLOPS MENA COMMUNITY

The DevOps to MLOps Transition Roadmap

The fastest career pivot in tech

DevOps → MLOps

⌚ 3–5 months

🕒 10–15 hrs/week

📖 31 free resources

👤 DevOps, SRE, platform and infrastructure engineers moving into ML.

[Read the original on LinkedIn ↗](#)



PHASE 1
Month 1



PHASE 2
Month 2



PHASE 3
Month 3



PHASE 4
Month 4

Over the last few months I've had a lot of conversations with DevOps engineers who feel stuck. They see the ML/AI wave, they know their infrastructure skills are valuable, but they don't know where to start.

Here's the honest truth: the DevOps to MLOps transition is one of the fastest career pivots you can make in tech. Faster than Software Engineer to MLOps, and much faster than Data Scientist to MLOps.



Why? Because you already own 60-70% of the required skillset. Docker, Kubernetes, CI/CD, cloud infrastructure, monitoring — you've got this. The missing piece is the ML-specific layer that sits on top.

I've helped a few DevOps engineers make this transition. This roadmap is what I wish I'd given them from day one. Timeline: 3-5 months at 10-15 hours/week if you're consistent.

First, Let's Talk About What You Already Have

Before you spend a dollar or an hour on new content, understand your existing advantage. As a DevOps engineer, you already have solid foundations you can skip entirely:

- **Linux, Bash, networking** — nailed it
- **Git, GitHub, branching strategies** — nailed it
- **Docker, container orchestration** — this is huge
- **Kubernetes basics** — even bigger
- **CI/CD pipelines** (Jenkins, GitHub Actions, GitLab CI) — you're ahead
- **Cloud platforms** (AWS/GCP/Azure) — you're way ahead
- **Prometheus, Grafana, log aggregation** — perfect
- **Infrastructure as Code** (Terraform, Ansible) — bonus points
- **Production mindset** — on-call, SLAs, incident response

If you compare this to the Basic MLOps Roadmap I published, you'll notice you can skip Phases 0, 1, and 3 almost entirely. That's already 4 months saved.

What you're missing is the ML layer and the MLOps-specific mindset. That's what this roadmap focuses on.



Phase 1 — Fill the Python & ML Gap (Month 1)

Most DevOps engineers I know are more comfortable in Bash, Go, or Shell scripting than in Python for data work. If that's you, don't skip this.

Python for ML (not general Python)

You don't need to master Python from scratch. You need to master the specific Python that ML engineers use daily: numpy, pandas, scikit-learn, and the general "data manipulation" mindset.

- [Real Python](#) — best focused articles on specific Python topics
- [freeCodeCamp's "Python for Data Science"](#)
- [Kaggle Learn Python](#) — short, hands-on

Machine learning fundamentals (concepts, not code)

You're not becoming a data scientist. You just need to understand what your data scientists are talking about: supervised vs unsupervised, train/val/test splits, overfitting, common metrics.

- [StatQuest with Josh Starmer](#) — best ML intuition on YouTube
- [Andrew Ng's "Machine Learning Specialization" \(Coursera, audit for free\)](#)
- [Google's Machine Learning Crash Course](#) — 15 hours, focused and free

Phase 1 project

Take a public dataset (Kaggle has thousands), train a simple scikit-learn model, evaluate it, and put the whole thing in a GitHub repo. It doesn't need to be sophisticated — it needs to exist.



Phase 2 — Learn ML-Specific Tooling (Month 2)

Here's where your DevOps skills pay off big. Every tool in this phase is another orchestration/infrastructure tool — just applied to ML. You'll pick them up faster than pure software engineers.

Data versioning — DVC

Think of DVC as Git for large files. Since you already know Git deeply, this is easy.

- [DVC official docs](#)
- [DVC YouTube channel](#)

Experiment tracking — MLflow

Think of MLflow as a "logging + database" system for ML experiments. If you've worked with observability tools, the mental model transfers directly.

- [MLflow docs](#)
- [freeCodeCamp's MLOps course](#) — 6 hours, ML-focused

Orchestration — Apache Airflow

You already know orchestration from your DevOps background. Airflow is orchestration for data pipelines. The concepts are identical to what you know; the vocabulary is different.

- [Marc Lamberti's YouTube](#) — the Airflow expert
- [Astronomer Academy](#) — free, with certification
- [Airflow official docs](#)

Model serving — FastAPI + BentoML

Since you know Docker, this is essentially "how ML people wrap their models for production."

The MLOps Zoomcamp

If you're going to take one course, take this. It's completely free, hands-on, and covers everything above in one structured path with a community.

- [MLOps Zoomcamp GitHub](#)
- [DataTalks.Club Slack](#)

Phase 2 project

Take your Phase 1 model, track experiments with MLflow, wrap it in FastAPI, containerize it with Docker (you already know how), and set up a simple Airflow DAG that retrains it weekly.

Phase 3 — Where Your DevOps Skills Shine (Month 3)

This is the phase where you'll suddenly realize: "I already know how to do most of this."

Kubernetes for ML

You already know Kubernetes. Now learn the ML-specific extensions.

- [KServe](#) — model serving on Kubernetes
- [Seldon Core](#)
- [Kubeflow](#) — full ML platform on K8s



GPU-specific Kubernetes

This is new territory for most DevOps engineers, but it builds on your existing K8s knowledge.

- [NVIDIA GPU Operator](#)
- **NVIDIA device plugin** — how GPUs are exposed to Pods
- **Node autoscaling with GPU nodes** — Karpenter is amazing here

CI/CD for ML (CI/CD/CT)

You know CI/CD. The new concept is CT — Continuous Training. Models need to be retrained automatically when data drifts or performance degrades. This is the piece that makes ML pipelines different from regular software pipelines.

- **GitHub Actions for ML workflows** — you already use GitHub Actions, just apply it to ML
- [CML \(Continuous Machine Learning\) by Iterative](#) — CI/CD extensions specifically for ML

Phase 4 — MLOps-Specific Concepts You Won't Know (Month 4)

This is the phase that separates a "DevOps engineer who deployed a model" from a real MLOps engineer. These concepts don't exist in traditional DevOps.

Data drift, concept drift, and model monitoring



In traditional software, a service is healthy if it responds correctly and doesn't error. In ML, a service can respond correctly, return errors, and still be unhealthy because the underlying model has silently degraded.

You need to understand:

- **Data drift** — the input distribution has shifted
- **Concept drift** — the real-world mapping between inputs and outputs has changed
- **Prediction drift** — the output distribution has shifted (an early warning sign)

Statistical tests you should understand:

- Kolmogorov-Smirnov test
- Population Stability Index (PSI)
- Jensen-Shannon divergence

Tools:

- [Evidently AI](#) — open-source, has excellent tutorials
- [Evidently YouTube channel](#)

Model registry

Different from container registries. A model registry tracks model versions, their training data, their performance metrics, and their deployment status.

- **MLflow Model Registry** — the default choice
- **BentoML Model Store** — if you use BentoML

Feature stores (later, not urgent)

You'll hear this term a lot. A feature store is essentially a database + API for ML features that ensures training-serving consistency. It's mostly relevant if you work at a company with dozens of ML models sharing features.

- [Feast](#) — open-source, most popular



Phase 4 project

Add monitoring to your Phase 2/3 pipeline. Set up drift detection with Evidently, alerts through your existing Prometheus/Grafana stack (you know how), and document a "what to do when drift is detected" runbook.

Phase 5 — Optional Specialization: LLMOps (Month 5+)

If you want to stay maximally relevant in 2026, add LLMOps to your DevOps + MLOps foundation. The demand is genuinely uncapped.

LLM serving

Your Kubernetes and containerization skills are directly applicable here. You just need to learn the ML-specific serving frameworks.

- [vLLM documentation](#) — the default LLM serving solution
- [Fast & Efficient LLM Inference with vLLM](#) — June 2026, free course

LLM observability

Think of this as observability for LLMs — traces, spans, costs, and quality metrics.

- [Langfuse](#) — open-source, self-hostable (I use this in production at Unifonic)
- [LangSmith](#) — commercial alternative

Basic understanding of LLM concepts



- [Andrej Karpathy's "Intro to LLMs" \(1 hour\)](#)
- [Hugging Face LLM Course](#)

Skills You Already Have That Will Set You Apart

Here's what most junior MLOps engineers coming from data science DON'T have — but you do:

- **Production incident management** — you know how to be on-call, how to lead post-mortems, how to build runbooks. This is gold. Most ML engineers have never touched a real production incident.
- **Cost optimization at scale** — you understand cloud billing, right-sizing, and spot instances. ML workloads are expensive and this expertise is rare among ML-native engineers.
- **Security & compliance** — networking, VPCs, IAM, secrets management. Compliance requirements around ML (PDPL, GDPR) are just standard security concerns applied to a new domain.
- **Automation mindset** — if you can automate it, you should. Most ML engineers automate their model training but not the infrastructure around it. That's your niche.
- **Observability discipline** — you already know that if it's not monitored, it doesn't exist. Apply this mindset to models and you'll be more valuable than most ML engineers.

Your Suggested 4-Month Learning Plan



1	Python + ML fundamentals	10-15
2	ML tooling (DVC, MLflow, Airflow, FastAPI)	10-15
3	Kubernetes for ML (KServe, GPU scheduling) + CI/CD/CT	10-15
4	Monitoring, drift detection, MLOps mindset	10-15
5+	LLMOps specialization (optional but recommended)	10-15

If you're consistent, you'll be MLOps-ready in 4 months and LLMOps-ready in 5-6 months. Way faster than any other transition path.

Common Mistakes DevOps Engineers Make in This Transition

Let me save you from mistakes I've seen others make.

- 1. Trying to become a data scientist first.** You don't need to master neural networks from scratch. You need to understand ML enough to build the infrastructure around it. Skip the Deep Learning Specialization, at least initially. Come back to it later if you want to specialize.
- 2. Underestimating the mindset shift.** Traditional DevOps says: "If it worked yesterday and nothing changed, it will work today." ML doesn't work that way. The data changes. The world changes. The model degrades even when nothing "broke."
- 3. Overinvesting in tools before understanding concepts.** Don't chase every new tool. The concepts (experiment tracking, model serving, drift detection) matter more than which specific tool you use.
- 4. Ignoring the ML people around you.** Learn from your data scientists and ML engineers. Ask them what breaks in their workflow. That's exactly the pain point you're solving.



Forgetting your existing strengths. Some transitions require understanding your past experience doesn't matter. This isn't a problem. You can turn your past experience into a competitive advantage. Lean into it.

العربية



Join the Community ↗



The MENA-Specific Reality

Since I'm building this community for our region, let me be direct:

- **The opportunity is massive.** Most companies in MENA don't have dedicated MLOps engineers yet — they have DevOps engineers being asked to "handle the ML stuff" without proper training. Being the person who intentionally transitions puts you ahead of the entire curve locally.
- **The salary jump is real.** Senior DevOps roles in the region cap out at a certain range. Senior MLOps roles start higher and grow faster. This transition is one of the highest-ROI career moves you can make right now.
- **Job titles vary wildly.** You might see the same role called "MLOps Engineer," "ML Platform Engineer," "AI Infrastructure Engineer," or even just "Senior DevOps with ML focus." Don't filter jobs by title alone — filter by responsibilities.

If this roadmap helped you, share it with a DevOps friend who's been thinking about this transition. The community grows when we help each other level up.

Other paths

Beginner → Job-ready

Basic MLOps Engineer Roadmap

From zero to job-ready

Mid → Senior

MLOps Roadmap for Seniors

Depth, system thinking, and production scale



JOIN US

Learning MLOps alone is

Thousands of engineers across MENA are already in the room — reviewing each other's code, sharing openings, and getting unblocked. It costs nothing.



WhatsApp

3,000+ members



LinkedIn

3,000+ followers



YouTube

All recordings



Discord

Chat & help



X

Follow for updates



Free MLOps and AI learning for engineers across the Middle East and North Africa — live sessions, structured roadmaps, open cohort courses, study groups, and mentorship. We also train, staff, and build for companies across the region.



MLOps MENA
COMMUNITY

العربية



[Join the Community ↗](#)



LEARN

[Roadmaps](#)

[Student Guides](#)

[Courses](#)

[The MLOps Practitioner](#)

[Sessions](#)

[Articles](#)

COMMUNITY

[Meet the team](#)

[Services for companies](#)

[FAQ](#)

[Contact us](#)

[Privacy Policy](#)

WHERE WE ARE

[WhatsApp](#)

[Discord](#)

[LinkedIn](#)

[X](#)

[YouTube](#)

[GitHub](#)

[Zomra](#)



Free to join this week with code **FREECORETEAM**

MLOps MENA
COMMUNITY

العربية



Join the Community ↗



© 2026 MLOps MENA Community. Free learning for the community.

Built by the community, for the community.